



리눅스 서버 사용 및 관리 설명서

version 1.01

A. Uthor

관리자 A. Nother

부관리자 T. Heother

Computer Science & Engineering

Engineering

Seoul National University

2026. 1. 17.

이 설명서는 비영리적 목적에 한해 제한 없이 배포할 수 있습니다.



서울대학교
SEOUL NATIONAL UNIVERSITY

Copyright ©2026 Seoul National University

[HTTPS://EN.SNU.AC.KR/INDEX.HTML](https://en.snu.ac.kr/index.html)

Compiled at March 13, 2026

Statement of Originality

I, the undersigned, declare that this is my own work unless where otherwise acknowledged and referenced.

Candidate A. Uthor

Signed _____

Date March 13, 2026

To The Researchers of Seoul National University

Calculation is the best way to prove.

Acknowledgements

acknowledgements

Preface

현대의 이공계 연구와 산업 현장은 더 이상 이론적 이해나 실험적 숙련도만으로 완결되지 않는다. 수치해석, 시뮬레이션, 데이터 분석, 자동화, 인공지능 및 고성능 계산(HPC)에 이르기까지, 코딩과 소프트웨어 개발 능력은 모든 이공계 분야에서 사실상 필수적인 연구 언어이자 사고 도구가 되었다. 물리학·화학·생명과학·재료과학·기계공학·전산학 등 학문적 경계를 막론하고, 연구의 설계, 수행, 검증, 재현성 확보의 핵심 단계마다 컴퓨터 기반 계산과 프로그래밍이 깊숙이 관여하고 있다.

그러나 이러한 중요성에도 불구하고, 많은 이공계 학생들은 코딩 그 자체보다도 개발 및 계산 환경을 구축하는 과정에서 큰 장벽을 경험한다. 운영체제별 차이, 컴파일러와 라이브러리의 의존성 문제, 패키지 충돌, 환경 변수 설정, 원격 서버 및 클러스터 사용, 버전 관리 시스템의 개념 등은 정규 교과과정에서 체계적으로 다루어지기 어렵다. 그 결과, 학생들은 “코드는 맞는데 실행이 되지 않는” 상황, “남의 컴퓨터에서는 되는데 내 환경에서는 안 되는” 문제, 혹은 “서버에서 무엇이 돌아가고 있는지 이해하지 못한 채 명령어를 복사·붙여넣기만 하는” 상태에 놓이기 쉽다. 이러한 환경적 불확실성은 학습 효율을 떨어뜨릴 뿐만 아니라, 연구 재현성과 협업 능력에도 부정적인 영향을 미친다.

이 문서는 바로 이러한 문제의식에서 출발한다. 본 문서는 특정 언어나 단일 소프트웨어의 사용법을 나열하는 안내서가 아니라, 이공계 학생이 연구와 개발을 수행하기 위해 반드시 이해해야 할 컴퓨터 및 서버 환경의 구조와 원리를 체계적으로 정리하는 것을 목표로 한다. 개발을 위한 Linux 환경 설정, 패키지 관리와 가상 환경의 개념, 컴파일과 실행의 흐름, 버전 관리 등을 단계적으로 다룸으로써, 독자가 단순한 사용자 수준을 넘어 자신의 계산 환경을 스스로 설계·관리할 수 있는 역량을 갖추도록 돕고자 한다.

이 문서를 통해 독자는 다음과 같은 것을 얻을 수 있을 것이다. 첫째, 개발 환경에서 발생하는 다양한 오류와 혼란을 “요령”이 아닌 구조적 이해를 바탕으로 진단하고 해결하는 능력. 둘째, 개인 컴퓨터에서 시작해 연구실 서버, 나아가 대규모 계산 자원까지 확장 가능한 일관된 작업 환경을 구축하는 사고방식. 셋째, 협업과 장기 연구에서 필수적인 재현 가능하고 유지보수 가능한 계산 환경을 설계하는 감각.

궁극적으로 이 문서는 코딩을 “해야만 하는 기술”이 아니라, 이공계 연구를 사고하고 확장하는 보편적 도구로 받아들이는 데 필요한 기초 체력을 제공하고자 한다. 환경 설정에 대한 이해는 눈에 잘 띄지 않지만, 연구 생산성과 독립성을 결정짓는 핵심 요소이다. 이 문서가 독자가 보다 안정적이고 자유로운 계산 환경 위에서 연구와 개발에 집중할 수 있는 발판이 되기를 바란다.

익명의 누군가

2026년 1월 17일, 관악사에서.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Windows vs. Linux	1
1.3	Ubuntu	1
1.4	WSL	1
2	Linux	2
2.1	Linux의 파일 시스템과 파일 관리	2
2.2	Bash script	5
3	Python Environments	10
3.1	주의사항	10
4	Compile Environments	11
5	Git	12
6	Server	13
	Appendix A Contributors	14
	References	15

List of Figures

List of Tables

List of Abbreviations

GUI Graphic User Interface

CLI Command Line Interface

Introduction

적당한 첫 문장

1.1 | Motivation

현대 이공계 연구에서 컴퓨터를 이용한 계산은 절대로 피할 수 없는 요소가 되었다. 특히, 대규모 데이터 처리, 복잡한 수치 계산, 시뮬레이션 등은 고성능 컴퓨팅 환경에서 수행되는 경우가 많다. 여러 대학에서도 기초 교양으로 컴퓨터 과목을 운영하고 있고, 컴퓨터공학과가 아닌 전공에서도 수업에서 프로그래밍이 요구되는 경우가 점점 늘어나고 있다. 그러나 대부분의 경우, 프로그래밍 자체는 가르쳐 주더라도 그 프로그래밍을 위한 적절한 환경 설정 방법을 잘 알려주지 않는다. 필자의 경우에도 새로 연구실에 들어온 학생이 리눅스 로컬 및 서버 환경 설정에서 큰 어려움을 겪거나, 학과 후배들이 프로그래밍 과제를 할 때 환경 설정부터 막히는 경우를 자주 보아왔다. 이에, 본 자료에서는 프로그래밍을 위한 환경 설정 방법을 가르쳐 주고자 한다. 본 자료에서는 독자가 컴퓨터에 대한 기초적인 지식은 가지고 있다고 가정한다.

1.2 | Windows vs. Linux

1.3 | Ubuntu

Ubuntu Linux는 윈도우와 같은 운영체제로, 윈도우와 달리 무료이며 공개 소스이다. 서버에 일반적으로 이용하는 Ubuntu Server는 GUI가 없는 Ubuntu로, 마우스 없이 명령줄만으로 제어하는 CLI 환경이다. 윈도우의 명령 프롬프트(cmd)와 유사하다.

1.4 | WSL

Linux

이 장은 Linux 명령어, bash script, .bashrc 설정, WSL과 VSCode 연결 및 설정 등에 대해 다룬다.

2.1 | Linux의 파일 시스템과 파일 관리

2.1.1 | Linux의 파일 시스템 구조

컴퓨터를 사용하기 위해서 가장 먼저 알아야 할 것은 무엇일까? 여러 가지가 있겠지만 파일이 어디에 있는지 알고, 그 위치로 이동해서 파일을 연 다음에 수정하고 저장하거나, 복사/이동/삭제 등을 할 줄 아는 것이 중요할 것이다. Windows와 같은 GUI 환경에서는 파일 탐색기를 통해 이런 작업을 할 수 있지만 CLI 환경에서는 명령어만으로 이러한 작업을 진행해야 하기 때문에 다소 어려움이 있을 수 있다.

일단 먼저 Linux의 파일 시스템 구조에 대해 알아보자. Windows의 경우, 디스크가 하나만 있다고 가정하면 모든 파일은 `C:\` 드라이브 아래에 존재한다. 하지만 Linux에서는 모든 파일이 `/` 라는 루트 디렉토리 아래에 존재한다. 그리고 그 아래에 여러 디렉토리가 존재하는 트리 구조로 되어 있다. 예를 들어, `/home/userid/dir1/dir2/file.txt` 라는 경로가 있다면, `/` 는 루트 디렉토리, `home` 은 루트 디렉토리 아래의 디렉토리, `userid` 은 `home` 디렉토리 아래의 디렉토리, `dir1` 은 `userid` 디렉토리 아래의 디렉토리, `dir2` 는 `dir1` 디렉토리 아래의 디렉토리, 그리고 `file.txt` 는 `dir2` 디렉토리 아래의 파일이다.

루트 디렉토리 아래에는 여러 다른 디렉토리들이 존재하는데, Linux 시스템의 기본적인 필수 파일들이 있는 디렉토리들도 있기 때문에 조심해야 한다. 이 문서의 수준에서는 보통 사용자가 파일을 저장하는 디렉토리인 `/home/userid` 아래에서 작업을 진행하게 될 것이다.

리눅스에서 파일이나 디렉토리를 지정할 때는 절대경로와 상대경로를 사용할 수 있다. 절대경로는 `/home/userid/dir1/dir2` 와 같이 루트 디렉토리에서 시작해서 내가 원하는 파일이나 디

렉토리의 절대적인 위치이다. 반면, 상대경로는 현재 내 위치에서 시작하는 경로다. 즉, 내가 지금 `/home/userid/dir1`에 있다면 여기서 `/home/userid/dir1/dir2`의 상대경로는 그냥 `dir2`다.

2.1.2 | 디렉토리 및 탐색

이제 파일 시스템 구조를 알았으니, 본격적으로 이를 탐색하는 방법에 대해 알아보자. Linux에서는 `cd` 명령어를 사용해서 디렉토리를 탐색할 수 있다.

`.`은 현재 디렉토리, `..`은 상위 디렉토리, `-`은 이전 디렉토리, `~`는 홈 디렉토리를 의미한다. `cd <dir>`라 입력하면 해당 디렉토리로 이동된다. 디렉토리를 표현할 때 절대경로, 상대경로 둘 다 가능하며, 실행시 주소쪽 값이 변경된다.

```
userid:~$ cd dir1
userid:~/dir1$ cd /home/userid/dir1
userid:/home/userid/dir1$ cd ..
userid:/home/userid$ cd -
userid:/home/userid/dir1$ cd ~
userid:~$
```

또한 `pwd` 명령어를 사용해서 현재 위치한 디렉토리를 확인할 수 있다.

다음으로 배울 명령어는 `ls` 명령어이다. 이 명령어는 현재 디렉토리에 있는 파일과 디렉토리 목록을 보여주는 명령어다. 만약 `~` 하위에 `dir1`이란 디렉토리랑 `file1.txt`라는 파일이 있다면, `ls` 명령어를 입력했을 때 다음과 같이 출력될 것이다.

```
userid:~$ ls
dir1  file1.txt
```

`ls -l` 명령어를 사용하면 파일과 디렉토리의 상세 정보를 볼 수 있다.

```
userid:~$ ls -l
drwxr-xr-x  2 userid userid 4096 Mar  6 14:26 dir1
-rw-r--r--  1 userid userid   0 Mar  6 14:26 file1.txt
```

각각의 열은 파일 또는 디렉토리의 권한, 링크 수, 소유자, 그룹, 크기, 마지막 수정 시간, 그리고 이름을 나타낸다. 권한이나 링크의 경우, 이 문서 수준에서 건드릴 일이 없고, 소유자, 파일 크기, 마지막 수정 시간이 중요할 것이다. 파일 크기는 바이트 단위로 표시된다.

`ls -a` 명령어를 사용하면 숨김 파일과 디렉토리도 볼 수 있다. 숨김 파일은 파일 명이 `.`으로 시작한다. 마지막으로 `ls -la` 명령어를 사용하면 상세 정보와 숨김 파일 모두 볼 수 있다.

2.1.3 | 파일 복사/이동/삭제

파일을 복사하려면 `cp` 명령어를 사용한다. 예를 들어, `file1.txt` 라는 파일을 `file2.txt` 로 복사하려면 다음과 같이 입력한다.

```
userid:~$ cp file1.txt file2.txt
```

이렇게 하면 현재 디렉토리에 있는 `file1.txt` 라는 파일이 `file2.txt` 라는 이름으로 복사된다. 만약 `file2.txt` 라는 파일이 이미 존재한다면, `cp` 명령어는 기존 파일을 덮어쓰게 된다. 만약 덮어쓰기를 방지하려면 `cp -i` 명령어를 사용하면 된다. 이 명령어는 덮어쓰기 전에 사용자에게 확인을 요청한다. 파일을 다른 디렉토리에 복사하고 싶으면 절대경로나 상대경로로 대상 디렉토리를 지정하면 된다. 예를 들어, `file1.txt` 를 `dir1` 디렉토리로 복사하려면 다음과 같이 입력한다.

```
userid:~$ cp file1.txt dir1/file2.txt
```

이렇게 하면 `file1.txt` 라는 파일이 `dir1` 디렉토리에 `file2.txt` 라는 이름으로 복사된다. `cp` 명령어는 디렉토리를 복사할 때는 `-r` 옵션을 사용해야 한다. 예를 들어, `dir1` 이라는 디렉토리를 `dir2` 라는 이름으로 복사하려면 다음과 같이 입력한다.

```
userid:~$ cp -r dir1 dir2
```

파일을 이동하려면 `mv` 명령어를 사용한다. 예를 들어, `file1.txt` 라는 파일을 `file2.txt` 로 이동하려면 다음과 같이 입력한다.

```
userid:~$ mv file1.txt file2.txt
```

이렇게 하면 `file1.txt` 라는 파일이 `file2.txt` 라는 이름으로 이동된다. `mv` 명령어도 `cp` 명령어와 비슷하게 덮어쓰기를 하며, 이를 방지할 수도 있고, 다른 디렉토리로 이동시킬 수도 있다. `mv` 명령어 역시 디렉토리를 이동시킬 수도 있으며, `cp` 와 달리 이때는 `-r` 옵션이 필요하지 않다.

마지막으로 파일을 삭제하려면 `rm` 명령어를 사용한다. 예를 들어, `file1.txt` 라는 파일을 삭제하려면 다음과 같이 입력한다.

```
userid:~$ rm file1.txt
```

이렇게 하면 현재 디렉토리에서 `file1.txt` 라는 파일이 삭제된다. 만약 디렉토리를 삭제하려면 `rm -r` 명령어를 사용해야 한다. 예를 들어, `dir1` 이라는 디렉토리를 삭제하려면 다음과 같이 입력한다.

```
userid:~$ rm -r dir1
```

`rm` 명령어는 삭제하기 전에 사용자에게 확인을 요청하는 `-i` 옵션도 제공한다. 주의: Linux에는 Windows와 달리 휴지통 개념이 없다. 즉, 파일이나 디렉토리를 삭제한 이후에는 복구하는 것이 쉽지 않다. 따라서 중요한 파일이나 디렉토리를 삭제할 때는 항상 주의해야 한다.

2.1.4 | 파일 만들기 및 열기

파일을 만들려면 `touch` 명령어를 사용한다. 예를 들어, `file1.txt` 라는 파일을 만들려면 다음과 같이 입력한다.

```
userid:~$ touch file1.txt
```

이렇게 하면 현재 디렉토리에 `file1.txt` 라는 이름의 빈 파일이 만들어진다.

파일을 열려면 여러 가지 방법이 있다. 가장 간단한 방법은 `cat` 명령어를 사용하는 것이다. 예를 들어, `file1.txt` 라는 파일의 내용을 보려면 다음과 같이 입력한다.

```
userid:~$ cat file1.txt
```

이렇게 하면 `file1.txt` 라는 파일의 내용이 터미널에 출력된다. 원래 `cat` 명령어는 concatenate에서 온 명령어로, 여러 파일을 붙이는 역할을 하는 명령어지만, 위와 같이 파일의 내용을 보기 위해서도 사용할 수 있다.

만약 파일이 너무 길어서 한 화면에 다 보이지 않는다면, `less` 명령어를 사용할 수 있다. 예를 들어, `file1.txt` 라는 파일의 내용을 페이지 단위로 보려면 다음과 같이 입력한다.

```
userid:~$ less file1.txt
```

이렇게 하면 `file1.txt` 라는 파일의 내용이 페이지 단위로 출력된다. `less` 명령어를 사용할 때는 엔터를 눌러서 다음 줄로, 스페이스바를 눌러서 다음 페이지로 넘어갈 수 있다. 또한 방향키로도 움직일 수 있다.

2.2 | Bash script

우리는 이전 장에서 간단한 Linux 명령어들을 배워보았다. 그런데 반복적인 작업을 해야 하거나, 명령어가 길어지면 매번 터미널에서 입력해서 하는 것은 번거로울 수 있다. 이럴 때는 bash script를 사용해서 여러 명령어들을 하나의 파일에 작성하고, 그 파일을 실행해서 한 번에 여러 명령어를 실행할 수 있다. Bash script는 일반적으로 `.sh` 확장자를 가진 파일로 작성된다. 이 장의 내용은 인파(2022)를 참고하였다.

늘 그렇듯이 “Hello, World!”를 출력하는 간단한 bash script를 만들어보자. 먼저 `hello.sh` 라는 파일을 만들고, 그 안에 다음과 같이 작성한다.

```
#!/bin/bash
echo "Hello, World!"
```

첫 번째 줄인 `#!/bin/bash` 는 이 파일이 bash script임을 나타내는 shebang 줄이다. 이 줄은 시스템에게 이 파일을 실행할 때 bash 셸을 사용하라고 지시하는 역할을 한다. 두 번째 줄인 `echo "Hello, World!"` 는 터미널에 “Hello, World!”라는 문자열을 출력하는 명령어이다.

이제 이 파일을 실행하려면 먼저 실행 권한을 부여해야 한다. 다음과 같이 입력한다.

```
userid:~$ chmod +x hello.sh
```

`chmod` 는 파일의 권한을 변경하는 명령어이다. 이 명령어를 사용하여 `hello.sh` 파일에 실행 권한을 부여할 수 있다. 이제 `hello.sh` 파일을 실행하려면 다음과 같이 입력한다.

```
userid:~$ ./hello.sh
Hello, World!
```

`./hello.sh` 는 현재 디렉토리에 있는 `hello.sh` 파일을 실행하라는 명령어이다. 이렇게 하면 `hello.sh` 파일이 실행되고, 터미널에 “Hello, World!”라는 문자열이 출력된다.

2.2.1 | 변수와 매개 변수

셸 스크립트에서는 다른 프로그래밍 언어와 비슷하게 변수를 사용할 수 있다. C언어나 파이썬의 변수와는 달리, bash의 변수는 모두 문자열로 저장되며 자료형을 따로 기입하지 않는다. 변수의 이름은 대/소문자를 구분하며 숫자를 포함할 수는 있지만 숫자로 시작해서는 안된다.

변수를 선언하려면 다음과 같이 입력한다.

```
#!/bin/bash
variable_name="value"
```

이렇게 하면 `variable_name` 이라는 이름의 변수가 만들어지고, 그 값은 `value` 라는 문자열이 된다. 변수를 사용할 때는 변수 이름 앞에 달러 기호(`\$`)를 붙여야 한다. 예를 들어, `variable_name` 이라는 변수를 출력하려면 다음과 같이 입력한다.

```
#!/bin/bash
variable_name="value"
echo $variable_name
```

이렇게 하면 `variable_name` 이라는 변수의 값인 `value` 가 터미널에 출력된다. 또한 변수의 값을 다른 변수에 할당할 수도 있다. 예를 들어, `new_variable` 이라는 변수에 `variable_name` 의 값을 할당하려면 다음과 같이 입력한다.

```
#!/bin/bash
variable_name="value"
```



```
new_variable=$variable_name
echo $new_variable
```

이렇게 하면 `new_variable` 이라는 변수에 `variable_name` 의 값이 할당되고, 터미널에 `value` 가 출력된다. 주의해야 할 점은 대입문자 `=` 양쪽에 공백이 있으면 안된다는 것이다. 예를 들어, `variable_name = "value"` 라고 입력하면 오류가 발생한다.

셸 스크립트에는 예약 변수가 존재한다. 이는 Linux 시스템에서 자동으로 설정되는 변수들로, 시스템 정보나 환경 설정 등을 담고 있다. 예를 들어, `\$HOME` 은 현재 사용자의 홈 디렉토리를 나타내는 예약 변수이다. `\$USER` 는 현재 사용자의 이름을 나타내는 예약 변수이다. 이러한 예약 변수들은 셸 스크립트에서 유용하게 사용할 수 있다.

마지막으로 매개 변수가 있다. 매개 변수는 셸 스크립트에 인자를 전달할 때 사용되는 변수들이다. `\$0` 은 실행된 셸 스크립트의 파일명이고, 그 이후 부터는 스크립트 넘겨진 매개 변수들이다. 예를 들어, `\$1` 은 첫 번째 매개 변수를 나타내고, `\$2` 는 두 번째 매개 변수를 나타낸다. 셸 스크립트를 실행할 때 인자를 전달하려면 다음과 같이 입력한다.

```
userid:~$ ./script.sh arg1 arg2
```

이렇게 하면 `script.sh` 라는 셸 스크립트가 실행되고, `arg1` 이라는 문자열이 `$1` 에, `arg2` 라는 문자열이 `$2` 에 할당된다. 셸 스크립트 내에서 `$1` 과 `$2` 를 사용해서 전달된 인자를 사용할 수 있다. 예를 들어, 다음과 같이 입력한다.

```
#!/bin/bash
echo "First argument: $1"
echo "Second argument: $2"
```

이렇게 하면 `script.sh` 라는 셸 스크립트가 실행될 때 전달된 첫 번째 인자와 두 번째 인자가 각각 출력된다. 전달한 인자의 개수는 `$#` 로 확인할 수 있다.

2.2.2 | 산술 연산

앞서 언급하였듯이, `bash`의 모든 변수는 기본적으로 문자열이기 때문에, 산술 연산을 하기 위해서는 별도의 구문을 사용하여야 한다. Bash에서 산술 연산을 할 수 있는 방법으로는 `expr` 명령어, `let` 명령어, 그리고 `$(( ))` 구문이 있다.

`expr` 명령어는 아래와 같이 사용한다.

```
#!/bin/bash
a=5
b=3
plus=`expr $a + $b`
```

```

minus='expr $a - $b'
multiply='expr $a \* $b'
divide='expr $a / $b'
remainder='expr $a % $b'

echo "plus: $plus"
echo "minus: $minus"
echo "multiply: $multiply"
echo "divide: $divide"
echo "remainder: $remainder"

```

중요한 점은 역따옴표로 감싸줘야 한다는 거, 피연산자와 연산자 사이에 공백이 필요한 거, 그리고 곱셈 문자는 `\` 로 이스케이프 처리해야 한다는 것이다. 또한 괄호 역시 이스케이프 처리를 해야 한다.

`let` 명령어는 아래와 같이 사용한다.

```

#!/bin/bash
a=5
b=3
let plus=a+b
let minus=a-b
let multiply=a*b
let divide=a/b
let remainder=a%b

echo "plus: $plus"
echo "minus: $minus"
echo "multiply: $multiply"
echo "divide: $divide"
echo "remainder: $remainder"

```

`$(( ))` 구문은 아래와 같이 사용한다.

```

#!/bin/bash
a=5
b=3
plus=$((a + b))
minus=$((a - b))
multiply=$((a * b))

```

```
divide=$((a / b))
remainder=$((a % b))

echo "plus: $plus"
echo "minus: $minus"
echo "multiply: $multiply"
echo "divide: $divide"
echo "remainder: $remainder"
```

Python Environments

이 장은 Python 기반의 프로그래밍을 위한 기본 연구 환경(conda와 pip 설치, 가상환경, 패키지 설치)을 다룬다.

3.1 | 주의사항

- **sudo reboot**은 그 어떠한 경우에도 사용하면 안 된다. 새로운 프로그램을 설치하는 등 작업 도중 블로그 글을 따라쓰게 될 수 있는데, 그중에 reboot 명령어가 있으면 건너뛰자. 타인의 프로세스가 비정상적으로 종료될 위험이 크다.
- 모든 사용자는 반드시 가상환경을 구성하고, 그 가상환경 내에 여러가지 연구에 필요한 프로그램들을 설치해 사용해야 한다. 이렇게 하면 관리자 권한 없이 원하는 대로 프로그램의 버전이나 설정을 바꿀 수 있다.

Compile Environments

이 장은 컴파일 언어 기반의 프로그래밍을 위한 기본 연구 환경을 다룬다. gcc와 gfortran 설치, Makefile, CMake 사용법 등을 포함한다.

Git

이 장은 Git과 GitHub을 이용한 버전 관리 방법을 다룬다.

Server

이 장은 원격 서버 연결 방법, VSCode 연결 방법, Slurm scheduler 사용법 등을 다룬다.

Contributors

김도영 dobby@snu.ac.kr 홍승주 sjhong6230@snu.ac.kr

References

인파. BASH 셸 프로그래밍 문법 마스터 총정리, 2022. URL https://inpa.tistory.com/entry/LINUX-셸-프로그래밍-핵심-문법-총정리#셸_스크립트_문법_검사_사이트. Accessed: 2026-03-10.